

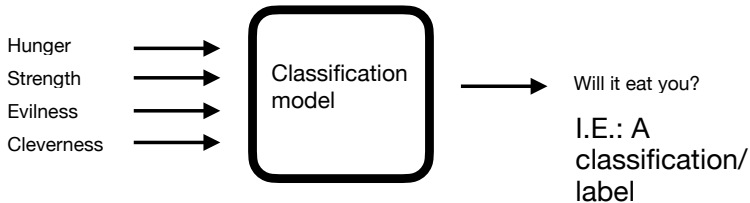
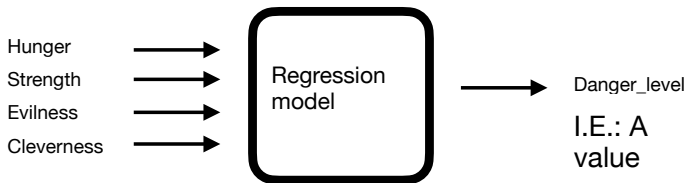
Text classification

(and also a bit about code refactoring)

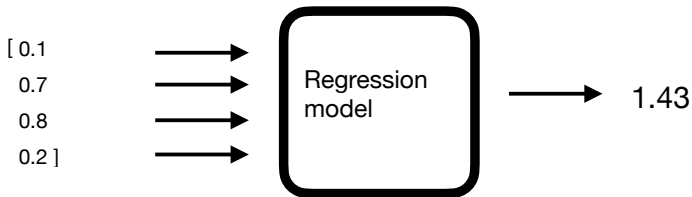
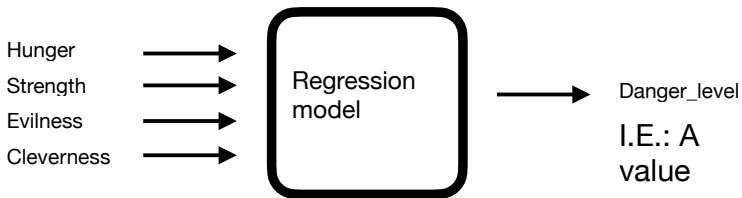
(CC-BY: Robert Ösling & Maria Skeppstedt)

1. Regression and Classification

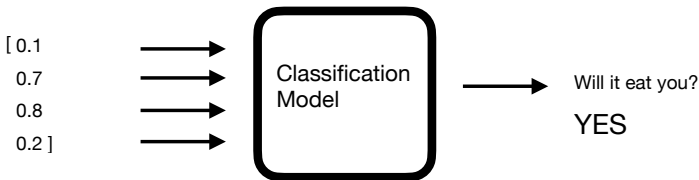
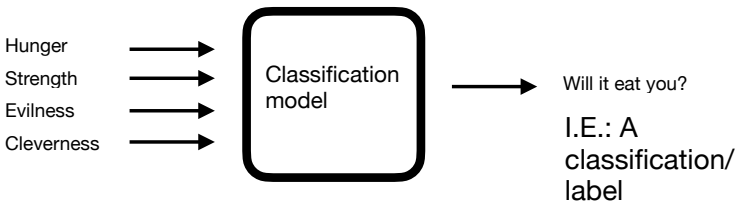
Difference between regression and classification



Regression predicts a value (e.g., the danger level), classification predicts a label (e.g. will it eat you? YES or NO.)



Regression predicts a value (e.g., the danger level).



Classification predicts a label (e.g. will it eat you? YES or NO.)

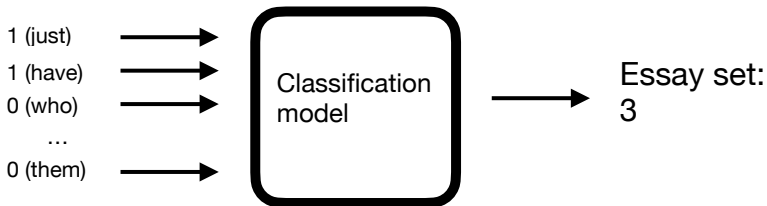
2. Classification

essay_set	essay
1	Dear local newspaper, I think effects computers have on people are great learning sk
1	Dear @CAPS1 @CAPS2, I believe that using computers will benefit us in many ways
1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more people use computers, but not ev
1	Dear Local Newspaper, @CAPS1 I have found that many experts say that computers
1	Dear @LOCATION1, I know having computers has a positive effect on people. The co
1	Dear @LOCATION1, I think that computers have a negative affect on us! How many p
1	Did you know that more and more people these days are depending on computers fo
1	@PERCENT1 of people agree that computers make life less complicated. I also agre
1	Dear reader, @ORGANIZATION1 has had a dramatic effect on human life. It has chan
1	In the @LOCATION1 we have the technology of a computer. Some say that the comp
1	Dear @LOCATION1, @CAPS1 people acknowledge the great advances that compute
1	Dear @CAPS1 @CAPS2 I feel that computers do take away from peoples life and are
1	Dear local newspaper I raed ur argument on the computers and I think they are a pos
1	My three detaileds for this news paper article is one state you opinion about the effe
1	Dear, In this world today we should have everyone useing computers. Computers ha
1	Dear @ORGANIZATION1, The computer blinked to life and an image of a blonde hair
1	Dear Local Newspaper I belive that computers have a negative effect on neoples liv

Classification task: Based on the words in the essay, predict the essay set. [1, 2, 3, 4, 5, 6, 7, 8]

The first set is essays about computers, the second about censorship in libraries, and so on.

- 2 I dont believe books should be taken off of the shelves for being offensive. Some books
- 2 Many people go to libraries; such as children, parents, and grandparents. I have come
- 2 There are many people that think certain books, music, movies, and magazines are c
- 2 I think that if there are books, music @CAPS1, movies, or magazines that we should
- 2 Going into a @CAPS1 everyone has their own opinions as to what they should see c
- 2 All libraries are not good cause some people are going to library and getting on the co
- 2 There are more than six billion people in the world. That means there are over six billi
- 2 What is offensive to you? See your answer might be different from the person sitting
- 2 I, personally, do not think that books, music, movies, or magazines should be remove
- 2 Why @CAPS1 @CAPS2? Is there movies, music, magazines, books, and more in c
- 2 I believe censorship should be used in libraries to a certain extent. There are too man
- 2 Have you ever been offended by a movie, book, song, or magazine? Did you have a
- 2 Libraries have tons of books from comedy all the way to violent. Some people like rea
- 2 Real life is rated @CAPS1 so why pretend it's not? If a library feels that if a book is to
- 2 There are many books in the library that shouldn't be there. From books about racism
- 2 Books can be good things, but are they always good? They teach us things, entertain
- 2 Just because I agree with something, do you have to agree too? Your cousin believ
- 2 offensive materials in libraries should not be taken off of the shelves because of their



Classification predicts a label

"I think this is good stuff useing lots of these words"

--> Label: 6

(Of the possible classes [1, 2, 3, 4, 5, 6, 7, 8])

"I believe life without internet
would be less productive"

--> Label: 1

(Of the possible classes [1, 2, 3, 4, 5, 6, 7, 8])

Example essay

Dear local newspaper, I think effects computers have on people are great learning skills/affects because they give us time to chat with friends/new people, helps us learn about the globe(astronomy) and keeps us out of troble! Thing about! Dont you think so? How would you feel if your teenager is always on the phone with friends! Do you ever time to chat with your friends or buisness partner about things. Well now - there's a new way to chat the computer, theirs plenty of sites on the internet to do so: @ORGANIZATION1, @ORGANIZATION2, @CAPS1, facebook, myspace ect. Just think now while your setting up meeting with your boss on the computer, your teenager is having fun on the phone not rushing to get off cause you want to use it. How did you learn about other countrys/states outside of yours? Well I have by computer/internet, it's a new way to learn about what going on in our time! You might think your child spends a lot of time on the

3. Code refactoring

The old version of the code needs to be **refactored**, to be able to use it with essay set as the classification target.

The behaviour of `construct_matrix` is not changed, so the code used calling this function for the regression model does not need to be changed.

In computer programming and software design, code refactoring is the process of restructuring existing source code—changing the factoring—without changing its external behavior. Refactoring is intended to improve the design, structure, and/or implementation of the software (its non-functional attributes), while preserving its functionality.

https://en.wikipedia.org/wiki/Code_refactoring

The behaviour of `construct_matrix` is not changed.

```
1 def construct_matrix(essays: list[dict]) -> tuple:
2     essay_features = [set(extract_features(essay["text"]))
3                         for essay in essays]
4     feature_counts = Counter(feature
5                               for features in essay_features
6                               for feature in features)
7     features = [feature
8                 for feature, count in feature_counts.items()
9                 if count >= 5]
10    feature_idx = \
11    {feature: i for i, feature in enumerate(features)}
12    coord_i, coord_j = [], []
13    for i, current_features in enumerate(essay_features):
14        for feature in current_features:
15            j = feature_idx.get(feature)
16            if j is not None:
17                coord_i.append(i) ; coord_j.append(j)
18    X = scipy.sparse.\
19        csr_array((np.ones_like(coord_i), (coord_i, coord_j)),
20                  shape=(len(essays), len(features)), dtype=np.float64)
21    y = np.array([essay["score"] for essay in essays], dtype=np.float64)
22    return X, y, features, feature_idx
```

We break out part of the code to a separate `matrix_from_features`, which can be called from `construct_matrix` used by the regression code, and that we also can use for the new classification code.

```
1 def construct_matrix(essays: list[dict]) -> tuple:
2     essay_features = [set(extract_features(essay["text"]))
3                         for essay in essays]
4
5     X, features, feature_idx = \
6         matrix_from_features(essay_features)
7
8     # The target values are dense, we use the essay scores directly
9     y = np.array([essay["score"] for essay in essays],\
10                  dtype=np.float64)
11
12     return X, y, features, feature_idx
```

Note that code calling `construct_matrix` does not have to care about that we changed it. From the callers perspective it looks the same.

```
1 def matrix_from_features(essay_features: list[set]) -> tuple:
2     feature_counts = Counter(feature
3                               for features in essay_features
4                               for feature in features)
5     features = [feature
6                 for feature, count in feature_counts.items()
7                 if count >= 5]
8     feature_idx = \
9     {feature: i for i, feature in enumerate(features)}
10
11     coord_i, coord_j = [], []
12     for i, current_features in enumerate(essay_features):
13         for feature in current_features:
14             j = feature_idx.get(feature)
15             if j is not None:
16                 coord_i.append(i) ; coord_j.append(j)
17
18     X = scipy.sparse.\
19         csr_array((np.ones_like(coord_i), (coord_i, coord_j)),
20                  shape=(len(essay_features), len(features)),
21                  dtype=np.float64)
22     return X, features, feature_idx
```

Now we can re-use `matrix_from_features`, `read_essay_sets` and `extract_features` as-is in our classification code:

```
1 def train_classifier(filename):
2     essay_classes = read_essay_sets(filename)
3
4     stop_words = stopwords.words('english')
5     features_all_classes, y = [], []
6     for nr, essays in essay_classes.items():
7         features_all_classes.extend(\
8             [set([f for f in extract_features(essay["text"])\
9                 if f not in stop_words])\
10              for essay in essays])
11         y.extend([nr] * len(essays))
12     X, features, feature_idx = \
13         matrix_from_features(features_all_classes)\
14         to_scikit_format(X)
15
16     clf = SGDClassifier(loss="log_loss")
17     clf.fit(X, y)
18     return clf, features, feature_idx
```

4. Implementing the classification

Here, we remove a list of pre-defined stop words from our features set. These are words (usually function words) that are usually not important for a text classifier. They come from the NLTK library.

```
1 def train_classifier(filename):
2     essay_classes = read_essay_sets(filename)
3
4     stop_words = stopwords.words('english')
5     features_all_classes, y = [], []
6     for nr, essays in essay_classes.items():
7         features_all_classes.extend(\
8             [set([f for f in extract_features(essay["text"])\
9                 if f not in stop_words])
10              for essay in essays])
11         y.extend([nr] * len(essays))
12     ...
13 from nltk.corpus import stopwords
14 print(stopwords.words('english'))
```

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't", 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', "couldn't", 'd', 'did', 'didn', "didn't" ...

X will contain the feature vector for each essay, and y will contain its essay-set number. (I.e., what we need to train our classifier).

```
1 def train_classifier(filename):
2     essay_classes = read_essay_sets(filename)
3     stop_words = stopwords.words('english')
4     features_all_classes, y = [], []
5     for nr, essays in essay_classes.items():
6         features_all_classes.extend(\
7             [set([f for f in extract_features(essay["text"])\
8                 if f not in stop_words])
9              for essay in essays])
10        y.extend([nr] * len(essays))
11    X, features, feature_idx = \
12        matrix_from_features(features_all_classes)
13    for x, yi in zip(X, y):
14        print(x.toarray(), yi, \
15              "\t Length of feature array:", len(x.toarray()))
```

```
[1. 1. 1. ... 0. 0. 0.] 1    Length of feature array: 8973
[1. 0. 1. ... 0. 0. 0.] 1    Length of feature array: 8973
[1. 0. 1. ... 0. 0. 0.] 1    Length of feature array: 8973
...
[0. 1. 1. ... 0. 0. 0.] 8    Length of feature array: 8973
[0. 1. 0. ... 0. 0. 0.] 8    Length of feature array: 8973
[0. 0. 1. ... 0. 0. 0.] 8    Length of feature array: 8973
[1. 1. 0. ... 0. 0. 0.] 8    Length of feature array: 8973
```

We now use another scikit-learn class: `SGDClassifier`, which is a (one of many) classes for training classification models.

```
1 def train_classifier(filename):
2     essay_classes = read_essay_sets(filename)
3
4     stop_words = stopwords.words('english')
5     features_all_classes, y = [], []
6     for nr, essays in essay_classes.items():
7         features_all_classes.extend(\
8             [set([f for f in extract_features(essay["text"])\
9                 if f not in stop_words])
10              for essay in essays])
11         y.extend([nr] * len(essays))
12     X, features, feature_idx = \
13         matrix_from_features(features_all_classes)
14     to_scikit_format(X) #Fix SciPy/scikit incompatibility
15
16     clf = SGDClassifier(loss="log_loss")
17     clf.fit(X, y)
18     return clf, features, feature_idx
```

Note that the method name and attribute for training (`fit(X, y)`) is the same as for the regression model.

```
1 def classification(filename):
2     clf, features, feature_idx = train_classifier(filename)
3
4     test_data = ["The computer is mostly positive, it's a exos
5     "Removing literature from the library puts us on a slippery
6     "A story about a cyclist cycling", \
7     "The story was about a girl not passing a test for a drive
8     "A story about someone praising their parents", \
9     "A story about something being built.", \
10    "A description of both being tolerant and the medical mean
11    "Laughing is a way of keeping us happy"]
12
13    X_test = get_test_features(test_data, \
14                               features, feature_idx)
15
16    y_pred = clf.predict(X_test)
17    print("Predicted:", y_pred)
```

Predicted: [1 2 3 4 5 4 7 8]

```
1 def classification(filename):
2     clf, features, feature_idx = train_classifier(filename)
3     test_data = ["The computer is mostly positive, it's a
4     ... "Laughing is a way of keeping us happy"]
5
6     X_test = get_test_features(test_data, \
7     features, feature_idx)
8     y_pred = clf.predict(X_test)
9     print("Predicted:", y_pred)
```

```
1 def get_test_features(texts, features, feature_idx):
2     X_test = scipy.sparse.dok_array(
3         (len(texts), len(features)), \
4         dtype=np.int32)
5
6     for i, text in enumerate(texts):
7         for feature in set(extract_features(text)):
8             j = feature_idx.get(feature)
9             if j is not None:
10                 X_test[i, j] = 1.0
11     X_test = X_test.tocsr()
12     to_scikit_format(X_test) # Fixing incompatibility
13     return X_test
```

```
1 def get_test_features(texts , features , feature_idx):
2     X_test = scipy.sparse.dok_array(
3         (len(texts), len(features)),\
4         dtype=np.int32)
5
6     for i, text in enumerate(texts):
7         for feature in set(extract_features(text)):
8             j = feature_idx.get(feature)
9             if j is not None:
10                 X_test[i, j] = 1.0
11     X_test = X_test.tocsr()
12     to_scikit_format(X_test) # Fixing incompatibility
13
14     for xi in X_test[:5]:
15         print(xi.toarray(), len(xi.toarray()))
16     return X_test
```

```
[0 0 0 ... 0 0 0] 8973
[0 0 0 ... 0 0 0] 8973
[0 0 0 ... 0 0 0] 8973
[0 0 0 ... 0 0 0] 8973
[0 0 0 ... 0 0 0] 8973
```

```
1 def classification(filename):
2     clf, features, feature_idx = train_classifier(filename)
3
4     test_data = ["The computer is mostly positive, it's a
5 ... "Laughing is a way of keeping us happy"]
6
7     X_test = get_test_features(test_data, \
8         features, feature_idx)
9     y_pred = clf.predict(X_test)
10    print("Predicted:", y_pred)
11
12    y_expected = np.array([1, 2, 3, 4, 5, 6, 7, 8])
13    print("Expected :", y_expected)
```

Predicted: [1 2 3 4 5 4 4 8]

Expected : [1 2 3 4 5 6 7 8]

SGDClassifier

```
class sklearn.linear_model.SGDClassifier(loss='hinge', *, penalty='l2',  
alpha=0.0001, l1_ratio=0.15, fit_intercept=True, max_iter=1000, tol=0.001,  
shuffle=True, verbose=0, epsilon=0.1, n_jobs=None, random_state=None,  
learning_rate='optimal', eta0=0.01, power_t=0.5, early_stopping=False,  
validation_fraction=0.1, n_iter_no_change=5, class_weight=None,  
warm_start=False, average=False)
```

[\[source\]](#)

Linear classifiers (SVM, logistic regression, etc.) with SGD training.

This estimator implements regularized linear models with stochastic gradient descent (SGD) learning: the gradient of the loss is estimated each sample at a time and the model is updated along the way with a decreasing strength schedule (aka learning rate). SGD allows minibatch (online/out-of-core) learning via the `partial_fit` method. For best results using the default learning rate schedule, the data should have zero mean and unit variance.

This implementation works with data represented as dense or sparse arrays of floating point values for the features. The model it fits can be controlled with the loss parameter; by default, it fits a linear support vector machine (SVM).

The regularizer is a penalty added to the loss function that shrinks model parameters towards the zero vector using either the squared euclidean norm L2 or the absolute norm L1 or a combination of both (Elastic Net). If the parameter update crosses the 0.0 value because of the regularizer, the update is truncated to 0.0 to allow for learning sparse models and achieve online feature selection.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

Some classifiers provide the functionality of increased verbose. They then show you what happens during training.

```
1 clf = SGDClassifier(loss="log_loss", verbose=1)
2 clf.fit(X, y)
```

The aim of the training is to minimize the loss (error made by the model), when classifying the training data. The **loss** decreases for every training **epoch** (at least in the beginning.). An epoch is one pass through the entire training data.

```
-- Epoch 1
Norm: 67.65, NNZs: 8973, Bias: -2.363849, T: 12976, Avg. loss: 0.272305
Total training time: 0.00 seconds.
-- Epoch 2
Norm: 37.22, NNZs: 8973, Bias: -2.409604, T: 25952, Avg. loss: 0.007143
-- Epoch 3
Norm: 25.66, NNZs: 8973, Bias: -2.422619, T: 38928, Avg. loss: 0.001835
-- Epoch 4
Norm: 19.62, NNZs: 8973, Bias: -2.436325, T: 51904, Avg. loss: 0.001317
-- Epoch 5
Norm: 16.11, NNZs: 8973, Bias: -2.448179, T: 64880, Avg. loss: 0.001476
-- Epoch 6
Norm: 13.92, NNZs: 8973, Bias: -2.460140, T: 77856, Avg. loss: 0.001726
-- Epoch 7
Norm: 12.48, NNZs: 8973, Bias: -2.472132, T: 90832, Avg. loss: 0.001934
-- Epoch 8
Norm: 11.51, NNZs: 8973, Bias: -2.483145, T: 103808, Avg. loss: 0.002142
```

Convergence after 8 epochs took 0.01 seconds

Starting training

0	1	2	3	4	5	6	...	158	...	272	...	330	...	y
just	have	buisness	feel	day	time	or	...	them	...	who	...	times	...	
1	1	1	1	1	1	1	...	0	...	0	...	1	...	1
0	0	1	0	0	0	1	...	1	...	0	...	1	...	1
0	0	0	0	0	0	1		0		1	...	0		1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0	0	0	0	0	0	0	0	0	0	0	0	0	0	8



w_0	w_1	w_2	w_3	w_4	w_5	w_6	...	w_158	w_272	...	w_230	...
?	?	?	?	?	?	?	...	?	?	...	?	...

Classification model

Training finished

0	1	2	3	4	5	6	...	158	...	272	...	330	...	y
just	have	buisness	feel	day	time	or	...	them	...	who	...	times	...	
1	1	1	1	1	1	1	...	0	...	0	...	1	...	1
0	0	1	0	0	0	1	...	1	...	0	...	1	...	1
0	0	0	0	0	0	1	...	0	...	1	...	0	...	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0	0	0	0	0	0	0	0	0	0	0	0	0	0	8



w_0	w_1	w_2	w_3	w_4	w_5	w_6	...	w_158	w_272	...	w_230	...
0.038	0.023	0.033	0.044	-0.06	0.070	0.05	...	-0.084	0.093	...	0.070	...

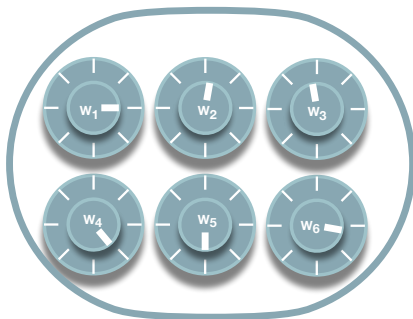
Classification model

SGD means **stochastic gradient descent**. This is an iterative method used for finding good values for $w_1, w_2 \dots w_n$ for the model. Small changes are made to $w_1, w_2 \dots w_n$ in each iteration, with the aim to decrease the loss.

```
1 clf = SGDClassifier(loss="log_loss", verbose=1)
2 clf.fit(X, y)
```

SGD means **stochastic gradient descent**. This is an iterative method used for finding good values for $w_1, w_2 \dots w_n$ for the model. Small changes are made to $w_1, w_2 \dots w_n$ in each iteration, with the aim to decrease the loss.

0	1	2	3	4	5	6	...	158	...	272	...	330	...	
just	have	buisness	feel	day	time	or	...	them	...	who	...	times	...	y
1	1	1	1	1	1	1	...	0	...	0	...	1	...	1
0	0	1	0	0	0	1	...	1	...	0	...	1	...	1
0	0	0	0	0	0	1	...	0	...	1	...	0	...	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0	0	0	0	0	0	0	0	0	0	0	0	0	0	8



```
1 clf = SGDClassifier(loss="log_loss", verbose=1)
2 clf.fit(X, y)
```

Different models can be trained using the `SGDClassifier`. What kind of model is trained is decided by the `loss` parameter. `log_loss` means it trains a logistic regression classifier.

The logistic regression classifier is then actually built on training a regression model for each class, and then applying the sigmoid function on the output, to get a probability score for each class ($\sigma(x) = \frac{1}{1+e^{-x}}$). This score is then used to select the most probable class.

5. Built in scikit-learn functionality for creating feature vectors from text

SciPy sparse matrices and scikit-learn seems to not be totally compatible. Needed to add this fix, to make it work:

```
1 def to_scikit_format(X):  
2     # https://github.com/scikit-learn/scikit-learn/issues/2986  
3     X.indices = \  
4         X.indices.astype(np.int32, casting='same_kind')  
5     X.indptr = X.indptr.astype(np.int32, casting='same_kind')
```

(It could be due to incompatible versions of the libraries or similar.)

Instead, you can use the `TfidfVectorizer` class available in `scikit-learn` to create the feature vectors from text. (Or the class `CountVectorizer`).

TfidfVectorizer

```
class sklearn.feature_extraction.text.TfidfVectorizer(*, input='content',
encoding='utf-8', decode_error='strict', strip_accents=None, lowercase=True,
preprocessor=None, tokenizer=None, analyzer='word', stop_words=None,
token_pattern='(?u)\\b\\w+\\b', ngram_range=(1, 1), max_df=1.0, min_df=1,
max_features=None, vocabulary=None, binary=False, dtype=<class
'numpy.float64'>, norm='l2', use_idf=True, smooth_idf=True,
sublinear_tf=False)
```

[\[source\]](#)

Convert a collection of raw documents to a matrix of TF-IDF features.

Equivalent to [CountVectorizer](#) followed by [TfidfTransformer](#).

Read more in the [User Guide](#).

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html